

Maklayn

Relatório técnico do projeto

Funcionamento, arquitetura, tecnologias e composição

Criador: Jhon Maklayn Mozer dos Santos

Data de início informada: 12/09/2026

Versão analisada: checkpoint 567c8ac

Relatório preparado: 07/10/2026

Documento técnico de referência do MVP Maklayn

Sumário

1	Resumo executivo	1
1.1	Leitura correta das métricas	1
2	Para que a Maklayn foi feita	1
2.1	Público e problemas atendidos	1
2.2	Princípios de resposta	2
3	Como o sistema funciona	2
3.1	Visão geral do fluxo	2
3.2	Chat inteligente	2
3.3	Copiloto de código	3
3.4	Arquivos grandes e ZIPs	3
3.5	Documentos, PDF e ZIP de projeto	3
4	Arquitetura técnica	3
4.1	Frontend	3
4.2	Backend	4
4.3	Dados e autenticação	4
4.4	Build e execução	4
5	Tecnologias e tipos de código usados	4
5.1	Linguagens e formatos	4
5.2	Principais tecnologias	5
6	Linhas por pasta	5
6.1	Principais arquivos da aplicação	6
7	Como o projeto foi construído	6
7.1	Etapa 1 — Base e decisão de produto	6
7.2	Etapa 2 — Identidade e interface	6
7.3	Etapa 3 — Inteligência e revisão	6
7.4	Etapa 4 — Arquivos e projetos	7
7.5	Etapa 5 — Qualidade e checkpoint	7
8	O que está pronto e o que falta	7
8.1	Entregue no MVP	7
8.2	Próxima fase recomendada	8
9	Conclusão	8

1 Resumo executivo

A Maklayn é uma plataforma web de inteligência artificial voltada para estudo, pesquisa, programação e organização de projetos. O produto foi concebido como uma bancada digital: em vez de oferecer apenas uma caixa de conversa, reúne módulos especializados para conversar, pesquisar, comparar respostas, escrever código, estudar para o ENEM e organizar projetos.

O MVP analisado foi construído sobre uma aplicação web full-stack em TypeScript, com frontend React servido por Vite, backend Express, procedimentos tRPC, integração com o LLM gerenciado da plataforma Manus e uma camada Drizzle preparada para banco de dados. A interface prioriza uma experiência editorial e operacional: barra lateral contextual, canvas amplo, editor de código, estados de carregamento, feedbacks visuais e modo claro/escuro.

A versão atual inclui, além do chat, um copiloto de código com dois agentes simultâneos, análise incremental de arquivos grandes, leitura de entradas de ZIP, anexos universais, exportação de PDF e criação de ZIP de projeto. A execução real de código em sandbox, a persistência privada de documentos e a pesquisa web ao vivo continuam explicitamente identificadas como próximas fases.

1.1 Leitura correta das métricas

As métricas deste relatório foram calculadas sobre os arquivos versionados pelo Git, excluindo `node_modules`, `dist`, `cache`s e artefatos locais. O número total inclui arquivos de configuração e o grande `pnpm-lock.yaml`; por isso, o relatório também apresenta o número mais útil para medir a aplicação: código executável e estilos da solução.

Métrica	Valor
Arquivos versionados	135
Linhas versionadas totais	23.792
Arquivos de aplicação (TS, TSX, CSS, JS, HTML, SQL)	112
Linhas de aplicação	13.931
Linhas do <code>pnpm-lock.yaml</code>	9.172
Último checkpoint analisado	567c8ac

2 Para que a Maklayn foi feita

A finalidade do projeto é oferecer um ambiente único para transformar perguntas e ideias em trabalho verificável. O nome Maklayn representa uma ferramenta de construção intelectual: o usuário fornece contexto, documentos, código ou um objetivo; a plataforma organiza a tarefa e retorna uma resposta com explicação, revisão e próximos passos.

2.1 Público e problemas atendidos

- **Estudantes:** organização de estudos, trilhas do ENEM, questões, feedback pedagógico e revisão de redação.
- **Pesquisadores:** síntese de hipóteses, organização de fontes e separação entre fato, inferência e incerteza.
- **Desenvolvedores:** geração, revisão, explicação, testes e auditoria defensiva de código.

- **Criadores de projetos:** transformação de um prompt em arquitetura, árvore de arquivos, implementação inicial, testes e instruções reproduzíveis.
- **Usuários que trabalham com documentos:** anexação de formatos variados, análise por partes e empacotamento em ZIP.

2.2 Princípios de resposta

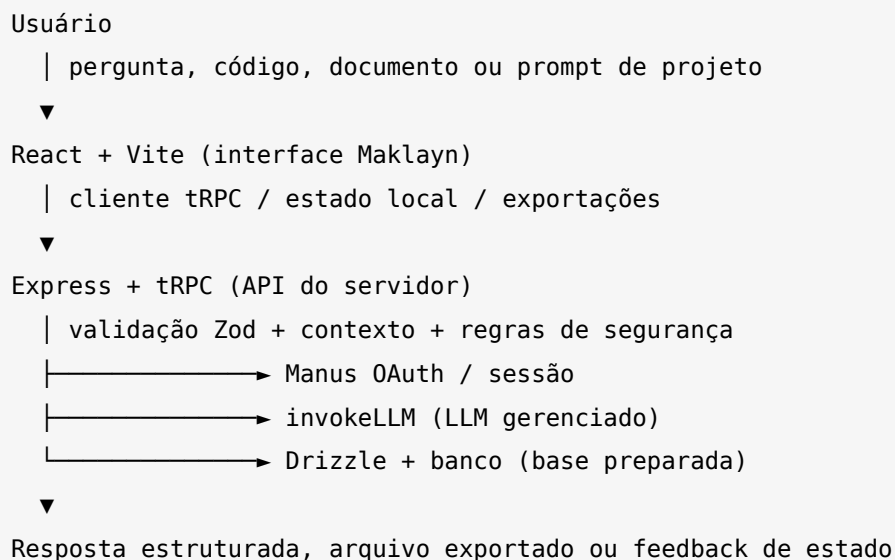
A Maklayn foi configurada para responder diretamente a pedidos permitidos, sem apresentar como verificado aquilo que não foi verificado. O comportamento esperado é:

1. entender a tarefa e o contexto fornecido;
2. separar fatos, inferências, opiniões e incertezas;
3. não inventar fontes, URLs, autores ou resultados;
4. adaptar profundidade e formato ao pedido;
5. quando houver risco técnico, apontar limitações e oferecer uma alternativa defensiva;
6. quando uma etapa depender de execução, compilação, rede ou credencial, declarar essa dependência.

3 Como o sistema funciona

3.1 Visão geral do fluxo

O fluxo principal da Maklayn é uma cadeia cliente-servidor. O navegador apresenta a interface, mantém o estado imediato da sessão e envia pedidos pelo cliente tRPC. O servidor recebe o pedido por uma rota tRPC, valida os dados com Zod, monta o contexto de sistema e chama o cliente LLM no backend. A resposta retorna ao navegador sem expor credenciais do provedor.



3.2 Chat inteligente

O procedimento `ai.chat` recebe uma lista de mensagens com papéis `user`, `assistant` ou `system`. O servidor acrescenta um prompt de sistema da Maklayn, encaminha o contexto ao LLM gerenciado e devolve `content`. Quando o serviço não responde, o frontend usa um modo demonstrativo explícito, sem fabricar citações ou fingir que uma resposta foi verificada.

3.3 Copiloto de código

O procedimento `ai.codeAssist` recebe operação, linguagem, tarefa, código e nome de arquivo. As operações disponíveis são:

- gerar código;
- revisar código;
- criar testes;
- explicar código;
- auditar segurança;
- montar projeto.

Para cada pedido, dois agentes são acionados em paralelo. O primeiro atua como implementador ou analista principal. O segundo faz uma revisão independente, procurando bugs, regressões, casos de borda, requisitos ausentes e testes que ainda precisam passar. A resposta final identifica as duas partes para que o usuário distinga implementação de revisão.

A operação de montagem de projeto pede uma arquitetura completa, árvore de arquivos, implementação inicial por arquivo, testes, variáveis de ambiente, comandos de instalação e checklist de produção. A Maklayn não afirma que criou ou executou os arquivos quando apenas gerou o conteúdo.

3.4 Arquivos grandes e ZIPs

Arquivos de texto e código são divididos em partes de aproximadamente 12.000 caracteres. Cada parte é enviada ao procedimento `ai.analyzeFileChunk`, que mantém um resumo acumulado. Dois agentes analisam cada parte: um leitor preserva estrutura, nomes e decisões; um auditor acrescenta riscos, bugs, segredos aparentes e perguntas abertas.

Quando o arquivo é ZIP, o frontend expande suas entradas com `fflate`, ignora diretórios vazios e identifica extensões binárias que não devem ser tratadas como texto. Entradas legíveis seguem o mesmo fluxo incremental. Entradas binárias são preservadas para exportação, ainda que a interpretação profunda exija um parser específico.

3.5 Documentos, PDF e ZIP de projeto

O CodeLab possui um campo universal de arquivos, sem filtro de extensão, e permite múltiplos anexos. O estado dos anexos é mantido na sessão do navegador. O botão PDF usa `jsPDF` para gerar um documento com o conteúdo atual ou o relatório da Maklayn. O botão ZIP usa `fflate` para criar `maklayn-project.zip` com:

- `maklayn-project/src/current-code.txt`;
- `maklayn-project/README.md`;
- `maklayn-project/maklayn-analysis.md`;
- `maklayn-project/attachments/`, contendo os anexos originais.

4 Arquitetura técnica

4.1 Frontend

O frontend está em `client/` e foi desenvolvido em React com TypeScript e TSX. `client/src/App.tsx` concentra o shell da aplicação, a navegação entre módulos, o estado local do MVP, o fluxo de autenticação e

a ligação das mutations tRPC. `client/src/components/CodeLab.tsx` implementa o editor, anexos, análise incremental, exportação de PDF e exportação de ZIP. `client/src/index.css` contém o sistema visual, responsividade, temas e estados de interação.

A interface usa componentes do ecossistema Radix UI, ícones lucide-react, sonner para feedbacks e streamdown para renderização de respostas Markdown. O tema claro usa superfícies marfim e destaques creme; o tema escuro usa destaques cinza.

4.2 Backend

O backend está em `server/` e é servido por Express. `server/_core/index.ts` inicia o servidor, integra Vite no desenvolvimento e expõe o health check. `server/_core/trpc.ts` configura o contexto e os procedimentos. `server/routers.ts` define autenticação, chat, copiloto de código e análise incremental de arquivos. `server/_core/llm.ts` encapsula a chamada ao LLM gerenciado.

A regra mais importante da separação é que credenciais e chamadas privilegiadas permanecem no servidor. O navegador recebe apenas o resultado necessário para apresentar a experiência.

4.3 Dados e autenticação

A pasta `drizzle/` contém schema, relações, configuração de migração e a migração inicial de usuários do starter. O fluxo de autenticação preservado é Manus OAuth, com sessão validada no backend. O MVP usa estado local para parte da experiência, enquanto tabelas completas para projetos, conversas, mensagens, fontes e documentos fazem parte da evolução planejada.

4.4 Build e execução

Os scripts principais são:

Comando	Finalidade
<code>pnpm dev</code>	Inicia o servidor de desenvolvimento com TypeScript em modo watch.
<code>pnpm check</code>	Executa o TypeScript compiler sem emitir arquivos.
<code>pnpm test</code>	Executa os testes Vitest do projeto.
<code>pnpm build</code>	Gera o bundle do frontend com Vite e do servidor com esbuild.
<code>pnpm start</code>	Executa o bundle de produção gerado.
<code>pnpm db:push</code>	Gera e aplica migrações Drizzle.
<code>pnpm db:migrate</code>	Aplica migrações já geradas.

5 Tecnologias e tipos de código usados

5.1 Linguagens e formatos

Extensão	Arquivos	Linhas
<code>.tsx</code>	67	9.358
<code>.ts</code>	41	3.420
<code>.json</code>	8	305
YAML	2	9.181

Extensão	Arquivos	Linhas
JavaScript	1	899
.md	3	152
sem extensão	7	182
.css	1	217
.html	1	24
.sql	1	13
.svg	1	6
.patch	1	28

Os valores de YAML são altos principalmente porque `pnpm-lock.yaml` registra a árvore de dependências e versões resolvidas. Ele é importante para reprodutibilidade, mas não equivale a código escrito especificamente para a lógica da Maklayn.

5.2 Principais tecnologias

- **TypeScript:** linguagem principal do frontend e backend, com tipos compartilhados.
- **TSX/React:** composição da interface, componentes, módulos e estados interativos.
- **Vite:** servidor e bundler do frontend.
- **Express:** servidor HTTP e integração com o runtime WebDev.
- **tRPC:** contrato tipado entre frontend e backend.
- **Zod:** validação dos inputs das procedures.
- **Drizzle ORM:** schema e migrações do banco.
- **Manus OAuth:** autenticação e sessão do starter.
- **LLM gerenciado:** geração de respostas e análise assistida no backend.
- **fflate:** leitura de ZIPs e criação do ZIP de projeto.
- **jsPDF:** geração de PDF no navegador.
- **Vitest:** testes automatizados existentes.
- **esbuild:** bundle do servidor para produção.
- **CSS customizado:** identidade visual, responsividade e temas claro/escuro.

6 Linhas por pasta

A tabela abaixo considera os arquivos versionados pelo Git. A pasta [raiz] contém configurações, documentação, lockfile e arquivos de projeto; por isso, seu total não deve ser lido como código de aplicação.

Pasta	Arquivos	Linhas	O que contém
client/	79	10.748	Interface React, páginas, componentes, estilos e manifesto público.
server/	25	2.837	Express, tRPC, autenticação, LLM, storage, helpers e testes.
drizzle/	6	165	Schema, relações, metadados e migração inicial.

Pasta	Arquivos	Linhas	O que contém
shared/	3	63	Tipos e constantes compartilhados entre cliente e servidor.
patches/	1	28	Patch de dependência.
[raiz]	21	9.951	Configurações, documentação, lock-file, Docker e arquivos de build.

6.1 Principais arquivos da aplicação

Arquivo	Linhas	Responsabilidade
client/src/ pages/ComponentShowcase.tsx	1.437	Showcase de componentes do starter.
client/src/components/ui/sidebar.tsx	733	Componente de navegação lateral reutilizável.
server/_core/llm.ts	453	Cliente do LLM gerenciado.
client/src/App.tsx	336	Shell da aplicação e módulos Maklayn.
client/src/components/CodeLab.tsx	208	Editor, copiloto, anexos, PDF e ZIP.
client/src/index.css	217	Sistema visual e responsividade.
server/routers.ts	aprox. 140	Procedures de chat, código e chunks de arquivo.

7 Como o projeto foi construído

7.1 Etapa 1 — Base e decisão de produto

O projeto partiu do template web com banco e servidor habilitados. A decisão foi manter React + Vite + Express + tRPC + Drizzle, evitando reescrever a infraestrutura do starter. O plano separou o que seria demonstrável no MVP do que exigiria serviços de produção, como RAG, storage privado, execução isolada e busca ao vivo.

7.2 Etapa 2 — Identidade e interface

A interface foi desenhada como uma bancada editorial: navegação lateral fixa, cabeçalho de contexto, módulos com propósito claro, cards assimétricos e áreas de trabalho. A marca começou com um símbolo tipográfico e depois recebeu a imagem medieval enviada pelo criador. A paleta foi ajustada para creme no tema claro e cinza no tema escuro.

7.3 Etapa 3 — Inteligência e revisão

O chat foi conectado ao invokeLLM no servidor com fallback transparente. Depois, o copiloto de código foi ampliado para dois agentes paralelos. O contrato passou a distinguir agente implementador e agente revisor, evitando que uma única resposta fosse apresentada como se tivesse sido testada quando não houve execução real.

7.4 Etapa 4 — Arquivos e projetos

A análise de documentos evoluiu de um campo de código para ingestão incremental. Textos são quebrados em chunks; ZIPs são abertos por entrada; anexos são mantidos na sessão; e o usuário pode exportar um pacote ZIP com código, documentação, relatório e arquivos originais. A geração de PDF foi adicionada no cliente com jsPDF.

7.5 Etapa 5 — Qualidade e checkpoint

Cada incremento foi submetido a `pnpm check`, `pnpm test`, `pnpm build`, health check e inspeção de diff. Os commits registrados mostram a evolução do MVP:

Commit	Data	Entrega
c71b05f	06/10/2026	Scaffold inicial.
0dec425	06/10/2026	MVP educacional Maklayn.
d1109d1	06/10/2026	Copiloto de programação.
ae1df14	06/10/2026	Respostas adaptativas e dois agentes.
5a69f11	06/10/2026	Arquivos grandes, ZIPs e blueprints.
bbae334	06/10/2026	Documentos, PDF e exportação ZIP.
d7ba9ec	07/10/2026	Campo universal de arquivos.
a8b7f71	07/10/2026	Nova logo medieval.
567c8ac	07/10/2026	Paleta creme e cinza.

A data de início 12/09/2026 é a data informada pelo criador neste relatório. O histórico Git disponível no ambiente começa com o scaffold em 06/10/2026; as duas datas representam, respectivamente, o início declarado do projeto e o início do histórico técnico recuperado no repositório.

8 O que está pronto e o que falta

8.1 Entregue no MVP

- dashboard responsivo com módulos de Chat, Pesquisa, Arena, Código, ENEM e Projetos;
- chat com integração LLM e fallback demonstrativo explícito;
- respostas de código geradas, revisadas e acompanhadas de testes sugeridos;
- modo de dois agentes em paralelo;
- análise incremental de textos e códigos grandes;
- leitura de ZIPs por entrada;
- campo universal para múltiplos arquivos;
- escrita de Markdown e código no editor;
- exportação de PDF e ZIP de projeto;
- autenticação Manus OAuth preservada;
- health check, manifesto de rotas, build e testes;
- temas claro e escuro e marca visual atualizada.

8.2 Próxima fase recomendada

1. persistir projetos, conversas, mensagens e anexos em banco e storage privado;
2. adicionar parsers dedicados para PDF, DOCX, XLSX, imagens, áudio e outros formatos;
3. implementar fila de indexação e busca semântica/RAG;
4. conectar executor isolado para compilar e testar código real com limites de CPU, memória, tempo e rede;
5. habilitar pesquisa web autorizada com referências rastreáveis;
6. completar controles de privacidade, exportação e exclusão de dados;
7. criar CI, observabilidade, backup e publicação de produção.

9 Conclusão

A Maklayn foi feita para ser uma estação de trabalho de IA, não apenas um chat. Sua arquitetura separa a interface, o servidor, a validação, o LLM, a autenticação e a futura persistência. O resultado atual é um MVP funcional e extensível: já demonstra os principais fluxos de estudo, pesquisa, programação, documentos e exportação, enquanto mantém declaradas as fronteiras que ainda exigem infraestrutura de produção.

O criador registrado neste documento é **Jhon Maklayn Mozer dos Santos**. A data oficial de início informada é **12/09/2026**.